

Dense mapping và biểu diễn (Dense Mapping and Representation)

PHỤ THUỘC A/01-hình-học-da-thi (triangulation, stereo); A/06-bat-dinh-va-lan-truyen-sai-so ($\sigma_z \approx z^2 \sigma_d / (bf)$) — quan hệ này quyết định chất lượng mọi bản đồ dày; C/13-quan-ly-ban-do (bài toán kích thước, ở đây gay gắt hơn nhiều bậc).

TÓM TẮT

Bản đồ thưa gồm vài nghìn điểm đủ để định vị nhưng không đủ để *làm gì với thế giới*. Một robot cần biết chỗ nào đi được; một hệ AR cần biết bề mặt ở đâu để đặt vật ảo lên; một hệ quét cần một mô hình có thể in ra. Chương này về các biểu diễn dày và về một câu hỏi mà ít ai hỏi thẳng: *ai cần chúng, để làm gì, và cái giá là bao nhiêu?* Nội dung có bốn phần. Một: lấy độ sâu dày từ ảnh — stereo, multi-view, và giới hạn vật lý mà không thuật toán nào vượt qua được. Hai: bốn họ biểu diễn cổ điển — TSDF, surfel, octree, mesh — với tiêu chí chọn dựa trên *việc bạn định dùng bản đồ để làm gì*, không dựa trên chất lượng hình ảnh. Ba: các biểu diễn học được — NeRF và Gaussian splatting — là hướng nóng nhất hiện nay, với một đánh giá thẳng thắn về chỗ chúng đã tốt và chỗ chưa. Bốn: một đánh đổi mà tài liệu hay bỏ qua. Nếu chỉ nhớ một điều: *bản đồ dày đẹp hơn nhưng độ chính xác tư thế thường không hơn bản đồ thưa* — và hiểu vì sao là hiểu được vị trí đúng của cả chương này trong một hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ thưa (sparse map)** — tập vài nghìn tới vài trăm nghìn điểm 3D rời rạc, mỗi điểm ứng với một đặc trưng ảnh. Đủ để định vị, không đủ để mô tả bề mặt.
- **bản đồ dày (dense map)** — biểu diễn mô tả *bề mặt* hoặc *thể tích* của cảnh, không chỉ vài điểm rời rạc. Tốn bộ nhớ hơn hàng trăm tới hàng nghìn lần.
- **TSDF** — *truncated signed distance function* — mỗi voxel lưu khoảng cách có dấu tới bề mặt gần nhất, cắt ở một ngưỡng. Bề mặt là tập mức không. Hợp nhất nhiều lượt quét bằng trung bình có trọng số.
- **surfel** — một mảnh bề mặt nhỏ có vị trí, pháp tuyến, bán kính và màu. Biểu diễn không cần lưới, nên dễ biến dạng khi bản đồ được hiệu chỉnh.
- **octree** — cấu trúc cây chia không gian thành tám phần đệ quy, chỉ chia sâu ở chỗ có nội dung. Tiết kiệm bộ nhớ cho không gian phần lớn là trống.
- **ESDF** — *euclidean signed distance field* — mỗi voxel lưu khoảng cách tới chướng ngại gần nhất. Đây là thứ mà bộ lập kế hoạch đường đi thật sự cần, không phải một mô hình đẹp.
- **NeRF** — *neural radiance field* — biểu diễn cảnh bằng một mạng ánh xạ từ (vị trí, hướng nhìn) sang (màu, mật độ), render bằng ray marching.
- **3D Gaussian Splatting** — biểu diễn cảnh bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn — nhanh hơn NeRF nhiều bậc vì không cần lấy mẫu dọc tia.

CÂU HỎI MỞ ĐẦU

- Q1.** Bản đồ dày dùng nhiều dữ liệu hơn bản đồ thưa hàng trăm lần. Vì sao nó thường *không* cho tư thế chính xác hơn?
- Q2.** Bộ lập kế hoạch đường đi của bạn cần biết chỗ nào đi được. Nêu biểu diễn nó thật sự cần, và nói vì sao một mesh đẹp không phải câu trả lời.
- Q3.** Stereo cho độ sâu ở mọi pixel có vấn. Nêu ba vùng mà nó chắc chắn thất bại, và nói vì sao đó là giới hạn vật lý chứ không phải giới hạn thuật toán.
- Q4.** 3D Gaussian Splatting cho hình ảnh đẹp hơn hẳn TSDF với cùng dữ liệu. Nêu ba lý do vì sao một robot sản phẩm năm 2026 có thể vẫn nên dùng TSDF.
- Q5.** Bạn có một bản đồ dày và loop closure vừa xảy ra, dịch chuyển các keyframe đi vài mét. Chuyện gì phải xảy ra với bản đồ dày, và vì sao đó là vấn đề khó hơn với TSDF so với surfel?

1.1 Đặt vấn đề

Mọi chương trước làm việc với bản đồ thưa: vài nghìn điểm 3D, mỗi điểm ứng với một góc hoặc một đốm trong ảnh. Bản đồ ấy đủ cho việc định vị — đó là toàn bộ nội dung Phần B — và nó rẻ.

Nó không đủ cho gần như mọi thứ khác. Một robot không thể lập kế hoạch đường đi từ một đám mây điểm thưa, vì giữa hai điểm có thể là không gian trống hoặc có thể là một bức tường và bản đồ không nói. Một hệ AR không thể đặt một vật ảo lên bàn nếu nó không biết mặt bàn ở đâu. Một hệ quét không thể xuất ra một mô hình.

Bài toán gốc: từ cùng những ảnh ấy, dựng một mô tả bề mặt hoặc thể tích của cảnh.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu rất khác nhau, và gộp chúng là một lỗi phổ biến vì nó dẫn tới việc chọn sai biểu diễn.

Nhóm một — điều hướng. Robot cần biết chỗ nào đi được. Yêu cầu: đúng về mặt an toàn (thà báo có chướng ngại khi không có còn hơn ngược lại), cập nhật nhanh, và ở một biểu diễn mà bộ lập kế hoạch dùng trực tiếp được. *Không cần đẹp.*

Nhóm hai — tương tác. AR, robot gấp vật. Cần bề mặt chính xác ở vùng quan tâm, với pháp tuyến. Cần đẹp ở mức đủ để không gây khó chịu.

Nhóm ba — tái tạo. Quét để lưu trữ, để in 3D, để dựng mô hình. Cần chính xác và đẹp; *không cần* thời gian thực.

Ba nhóm ấy dẫn tới ba lựa chọn biểu diễn khác nhau, và phần lớn nhầm lẫn trong lĩnh vực này đến từ việc đọc một bài báo viết cho nhóm ba rồi áp dụng cho nhóm một.

Có một câu hỏi thứ hai, ít được hỏi thẳng và quan trọng hơn: *bản đồ dày có giúp việc định vị không?* Trực giác nói có — nhiều dữ liệu hơn thì tốt hơn. Mục 4 lập luận rằng câu trả lời chủ yếu là *không*, và hiểu vì sao là hiểu được vị trí đúng của cả chương này.

TRỰC GIÁC

Hình dung sự khác nhau giữa một tấm bản đồ du lịch và một mô hình sa bàn.

Bản đồ du lịch có vài chục ký hiệu: nhà thờ ở đây, ga tàu ở kia, ngã tư ở đó. Nó đủ để bạn biết mình đang ở đâu và đi hướng nào. Nó nhẹ, bạn gấp bỏ túi được, và nó không nói gì về việc vỉa hè cao bao nhiêu centimet.

Sa bàn thì mô phỏng cả thành phố — từng bức tường, từng bậc thềm. Nó nặng, tốn công dựng, nhưng nó trả lời được những câu mà bản đồ không trả lời được: cái xe đẩy của tôi có qua được chỗ này không? Cái kệ này có đủ chỗ đặt vật kia không?

Hai thứ phục vụ hai mục đích, và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn bản đồ*. Để biết mình đang ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận. Cả một bức tường phẳng của sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Đó là toàn bộ mục 4 của chương, phát biểu bằng hình ảnh: thông tin về *vị trí* nằm ở những chỗ *khác biệt*, và bản đồ thưa đã lấy hết những chỗ ấy rồi. Phần dày thêm vào là phần *không khác biệt* — hữu ích để biết hình dạng thế giới, vô ích để biết mình ở đâu trong đó.

Còn một chuyện nữa. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ cho đúng (loop closure), thì với tấm bản đồ du lịch, bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải *làm lại* cả một vùng — và đó là lý do bản đồ dày khó sống chung với SLAM hơn nhiều so với vẽ ngoài.

1.2 Lấy độ sâu dày

A. Stereo và giới hạn vật lý

Với cặp stereo đã hiệu chỉnh, bài toán là tìm disparity cho *mọi* pixel. Khác với B/07, nơi ta chọn vài trăm điểm tốt, ở đây ta phải xử lý cả những pixel không có gì đặc biệt.

SGM [1] là phương pháp cổ điển vẫn được dùng rộng rãi, và ý tưởng của nó đáng biết. Bài toán đúng là tối ưu một hàm năng lượng hai chiều với số hạng dữ liệu (độ khớp) và số hạng trơn (disparity của pixel cạnh nhau nên gần nhau) — nhưng tối ưu năng lượng 2D là NP-khó. SGM xấp xỉ bằng cách tối ưu chính xác dọc *nhiều đường một chiều* qua mỗi pixel rồi cộng chi phí lại. Rẻ, song song hoá tốt, và chất lượng đủ cho hầu hết ứng dụng.

Ba vùng mà stereo *chắc chắn* thất bại, và đây là câu trả lời cho câu hỏi mở đầu thứ ba — điều quan trọng là cả ba là giới hạn *vật lý*, không phải giới hạn thuật toán.

Một — vùng không có vân. Một bức tường trắng: mọi pixel trong một hàng trông giống hệt nhau, nên không có thông tin nào xác định disparity. Không thuật toán nào tạo ra thông tin từ chỗ không có; cái tốt nhất có thể làm là *nội suy* từ biên và *thừa nhận* rằng đó là nội suy.

Hai — vùng bị che khuất. Một phần cảnh nhìn thấy từ camera trái nhưng bị che với camera phải. Không có tương ứng nào tồn tại. Phát hiện được bằng kiểm tra trái-phải, và câu trả lời đúng là đánh dấu “không biết”.

Ba — bề mặt phản chiếu hoặc trong suốt. Gương, kính, nước, kim loại bóng. Ở đây tệ hơn hai trường hợp trên: khớp điểm *thành công* nhưng nó khớp với ảnh phản chiếu, nên độ sâu thu được là độ sâu của một vật không tồn tại ở đó. Đây là kiểu hỏng im lặng, và nó là lý do robot đâm vào cửa kính.

Ngoài ba vùng ấy, có một giới hạn định lượng chi phối mọi thứ: $\sigma_z \approx z^2 \sigma_d / (bf)$ từ A/06. Nó nói rằng dù thuật toán hoàn hảo, chất lượng độ sâu suy giảm theo bình phương khoảng cách. Một bản đồ dày ở xa là một bản đồ dày *sai*, và vẽ nó ra trông vẫn đẹp.

B. Multi-view và độ sâu học được

Với nhiều khung thay vì hai, ta có multi-view stereo. Lợi ích: nhiều đường cơ sở khác nhau nên nhiều tầm độ sâu được phục vụ tốt; và nhiều góc nhìn nên ít bị che khuất hơn. PatchMatch stereo và các biến thể là chuẩn cho tái tạo ngoại tuyến.

Độ sâu học được (B/08 mục 4C) giải quyết được vấn đề vùng không có vân — vì mô hình mang tiên nghiệm về việc các bề mặt thường trông thế nào. Đây là một đóng góp thật và là chỗ học máy giúp rõ ràng.

Cảnh báo vẫn như ở B/08: kết quả *trông* chắc chắn ở mọi pixel, kể cả những pixel mà hình học không nói gì. Một bản đồ dày hoàn toàn từ độ sâu học được là một bản đồ mà một phần là phép đo và một phần là phỏng đoán, *không có gì phân biệt hai phần*. Với ứng dụng an toàn — robot tránh chướng ngại — đó là điều nguy hiểm. Cách xử lý đúng: giữ hai kênh, một cho giá trị và một cho nguồn gốc (đo được hay suy ra), và để bộ lập kế hoạch đối xử với chúng khác nhau.

1.3 Bốn họ biểu diễn cổ điển

Bảng 1.1. Bốn họ biểu diễn. Cột “dùng cho” là tiêu chí chọn đúng — chọn theo việc bạn định làm gì với bản đồ, không theo chất lượng hình ảnh.

Biểu diễn	Cơ chế	Dùng cho	Hỏng ở
TSDF	Voxel lưu khoảng cách có dấu; bề mặt là tập mức không; hợp nhất bằng trung bình có trọng số	Tái tạo bề mặt chất lượng cao; quét trong nhà	Bộ nhớ theo thể tích; khó biến dạng khi loop closure
Surfel	Các mảnh bề mặt rời có vị trí, pháp tuyến, bán kính	Khi bản đồ phải <i>biến dạng</i> — ElasticFusion	Không có bề mặt liền; khó truy vấn “điểm này trông hay đặc”
Octree / occupancy	Chia không gian đệ quy; mỗi ô lưu xác suất bị chiếm	Điều hướng — trả lời trực tiếp “chỗ này đi được không”	Không mô tả bề mặt chính xác; độ phân giải hữu hạn
Mesh	Lưới tam giác	Hiển thị, in 3D, mô phỏng vật lý	Đắt để cập nhật gia tăng; khó xử lý topology thay đổi

Hai điểm đáng nói kỹ hơn bảng.

A. Điều hướng cần ESDF, không cần mesh

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những nhầm lẫn tốn kém nhất trong lĩnh vực.

Bộ lập kế hoạch đường đi không cần biết bề mặt trông thế nào. Nó cần trả lời hai câu: *điểm này có trống không?* và *điểm này cách chướng ngại gần nhất bao xa?* Câu thứ hai đặc biệt quan trọng vì phần lớn bộ lập kế hoạch hiện đại dùng một hàm chi phí dựa trên khoảng cách tới chướng ngại, và chúng cần cả *gradient* của hàm đó.

ESDF trả lời trực tiếp cả hai. Một mesh đẹp thì không — để trả lời “cách chướng ngại bao xa” từ một mesh, bạn phải tính khoảng cách tới tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ lập kế hoạch là quá đắt.

Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*. Một bản đồ occupancy độ phân giải thô mà trả lời truy vấn trong micro giây hữu ích hơn một mesh hoàn hảo mà mỗi truy vấn tốn mili giây.

B. Vấn đề biến dạng khi loop closure

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là vấn đề kỹ thuật khó nhất của bản đồ dày.

Khi loop closure xảy ra (B/11), các keyframe dịch chuyển — có thể vài mét. Bản đồ thưa xử lý dễ: mỗi landmark gắn với một keyframe neo, nên nó dịch theo. Bản đồ dày thì khó, và mức khó phụ thuộc vào biểu diễn.

Với TSDF, dữ liệu được *hợp nhất* vào một lưới voxel cố định trong không gian. Sau khi các keyframe dịch, lưới ấy không còn đúng — và không có cách nào “dịch” một lưới voxel đã trộn dữ liệu từ nhiều góc nhìn. Cách duy nhất đúng là *tích hợp lại từ đầu* từ các khung gốc, tức phải lưu toàn bộ ảnh độ sâu, tức tốn bộ nhớ khổng lồ. Các cách thoả hiệp: chia bản đồ thành các khối gắn với keyframe (mỗi khối dịch theo keyframe của nó, chấp nhận các đường nối không khớp), hoặc chỉ tích hợp lại vùng bị ảnh hưởng nhiều nhất.

Với surfel, mỗi surfel gắn với một keyframe nên nó dịch theo một cách tự nhiên — đây chính là lý do ElasticFusion [2] chọn surfel. Cái giá là mất bề mặt liền và khó trả lời truy vấn thể tích.

Đây là một ví dụ rõ về việc *lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc tới*. Nếu bạn đọc một bài về TSDF mà không nghĩ tới loop closure, bạn sẽ chọn nó rồi phát hiện vấn đề sau nhiều tháng.

1.4 Biểu diễn học được: ảnh chụp tại 9/2026

CẠM BÃY

Mục này chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất của cả cây. Lĩnh vực này chuyển động rất nhanh; mọi đánh giá dưới đây nên được rà lại sau sáu tháng.

Ngoài ra, tôi **không dẫn con số so sánh chất lượng** giữa các phương pháp. Lý do như ở B/07: các chỉ số (PSNR, độ chính xác bề mặt) phụ thuộc mạnh vào bộ dữ liệu và vào cách đo, và các bài báo trong lĩnh vực này thường so với baseline được cài lại chứ không phải số công bố gốc — đúng cỡ đồ ở `research-rules.md` mục 4. Điều tôi đưa ra là đánh giá về *cơ chế* và về *mức trưởng thành cho sản phẩm*, và cả hai đều là phán đoán của tôi chứ không phải kết quả đo.

A. NeRF-SLAM

iMAP [3] và NICE-SLAM [4] thay biểu diễn tường minh bằng một mạng: cảnh là các trọng số của mạng, và cả tư thế lẫn trọng số được tối ưu cùng lúc bằng phần dư quang trắc — về bản chất là một phương pháp direct (B/08) với một biểu diễn cấu trúc rất khác.

Cái được: biểu diễn liên tục, nội suy tự nhiên vào vùng chưa quan sát, bộ nhớ không tỉ lệ với thể tích.

Cái mất, và đây là ba điểm quyết định cho sản phẩm: *chậm* — render cần lấy mẫu nhiều điểm dọc mỗi tia; *quên* — một mạng huấn luyện tăng dần có xu hướng quên vùng cũ khi học vùng mới, và cách chữa (giữ lại các khung cũ để huấn luyện lại) làm mất lợi thế bộ nhớ; và *không có bảo đảm* — không cách nào biết một vùng trong biểu diễn là phép đo hay là ảo giác nội suy.

B. 3D Gaussian Splatting

3DGS [5] thay mạng bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn theo thứ tự độ sâu. Điểm quyết định: *không cần lấy mẫu dọc tia*, nên nhanh hơn NeRF nhiều bậc trong khi chất lượng hình ảnh tương đương hoặc hơn.

SplaTAM [6] và MonoGS [7] đưa nó vào SLAM: Gaussian là bản đồ, và tư thế được ước lượng bằng cách tối ưu phần dư quang trắc qua render khả vi.

Đây là hướng nóng nhất tại thời điểm chốt, và lý do dễ hiểu: nó cho hình ảnh đẹp hơn mọi biểu diễn cổ điển với cùng dữ liệu, và nó đủ nhanh để nghĩ tới thời gian thực.

Ba lý do vì sao một robot sản phẩm năm 2026 vẫn có thể nên dùng TSDF — đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Một — truy vấn. 3DGS tối ưu cho việc *render*, không cho việc trả lời “điểm này có trống không”. Một đám Gaussian không phải một hàm occupancy, và trích xuất bề mặt hoặc ESDF từ nó là một bước thêm, không rẻ và không được bảo đảm. Với điều hướng, đây là điểm chặn.

Hai — bộ nhớ và phần cứng. Hàng triệu Gaussian, mỗi cái vài chục tham số, cộng với việc render cần GPU. Với một robot dịch vụ chạy trên SoC nhúng, đây là ngân sách không có.

Ba — tính dự đoán được và độ trưởng thành. Chế độ hồng của TSDF đã được hiểu rõ sau mười lăm năm; chế độ hồng của 3DGS-SLAM thì chưa. Với sản phẩm, đây là lập luận nặng ký, và nó là cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07 mục 2B.

Đánh giá thẳng thắn của tôi tại 9/2026: 3DGS là một bước tiến thật và rất có thể là tương lai của biểu diễn cảnh; nó *chưa* là lựa chọn cho một robot điều hướng chạy trên phần cứng hạn chế. Với nhóm ba — tái tạo ngoại tuyến — nó đã rất mạnh. D/22 xếp “3DGS cho localization thực dụng” làm một trong tám khoảng trống, và lý do nằm ở ba điểm trên.

1.5 Đánh đổi bị bỏ qua

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là mục mà tôi cho là có giá trị cao nhất của chương.

Bản đồ dày thường không cho tư thế chính xác hơn bản đồ thưa. Điều này phản trực giác và đáng dẫn ra lý do.

Thông tin về tư thế nằm ở đâu? Quay lại Jacobian ở A/03: một quan sát đóng góp vào ma trận thông tin tỉ lệ với *gradient* của hàm đo. Với phần dư quang trắc, pixel không có gradient đóng góp *đúng bằng không* (B/08 mục 2B). Với phần dư chiếu, một điểm không phân biệt được với lân cận của nó không cho tương ứng đáng tin.

Nói cách khác: *thông tin về tư thế tập trung ở những chỗ có cấu trúc*, và một front-end thưa tốt đã lấy gần hết những chỗ ấy. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết, nhưng chúng không thêm ràng buộc mạnh cho tư thế.

Có ngoại lệ và đáng nêu rõ để công bằng. Trong môi trường rất nghèo đặc trưng — một hành lang trắng có gradient rất yếu nhưng khác không — direct method dày khai thác được phần thông tin ít ỏi mà front-end thưa bỏ qua, vì detector cần một cực đại rõ ràng để tồn tại. Đây là lợi thế thật của direct, đã nêu ở B/08 mục 2A. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — hai thứ khác nhau: ta có thể dùng phần dư dày để ước lượng tư thế mà vẫn chỉ lưu một bản đồ thưa.

Kết luận thiết kế, và nó gọn: *xây bản đồ dày vì bạn cần nó cho một truy vấn cụ thể — điều hướng, tương tác, hiển thị — không phải vì bạn kỳ vọng nó cải thiện định vị.* Nếu mục tiêu là định vị chính xác hơn, hãy đầu tư vào front-end (B/07), vào hiệu chuẩn (C/14), hoặc vào quản lý bản đồ (C/13) — ba chỗ đó cho lợi ích lớn hơn nhiều trên mỗi đơn vị công sức.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OctoMap [8]	Octree occupancy có xác suất; truy vấn nhanh	Lựa chọn mặc định cho điều hướng; mục 3A
KinectFusion [9]	TSDF thời gian thực trên GPU	Công trình khai sinh dòng TSDF; đọc để hiểu cơ chế hợp nhất
ElasticFusion [2]	Surfel để bản đồ biến dạng được khi loop closure	Mục 3B — lựa chọn biểu diễn bị chi phối bởi yêu cầu hệ thống
NICE-SLAM [4]	NeRF phân cấp cho SLAM	Mục 4A; chú ý vấn đề quên và tốc độ
SplaTAM [6], MonoGS [7]	3DGS-SLAM với render khả vi	Mục 4B; ảnh chụp tại 9/2026, rà lại sau sáu tháng
SGM trong OpenCV	Stereo dày cổ điển, chạy được trên CPU	Mục 2A; vẫn là baseline đáng tin cho sản phẩm

MINI PROJECT SLAM · Đo xem bản đồ dày có cải thiện định vị không — trên dữ liệu của chính bạn · 3 ngày

Mục tiêu	kiểm mệnh đề ở mục 5 thay vì tin nó, và đồng thời đo cái giá thật của bản đồ dày.
Dữ liệu	một chuỗi RGB-D có chân trị (TUM-RGBD), cộng một chuỗi tự quay trong môi trường <i>nghèo đặc trưng</i> — một hành lang trắng, một phòng trống. Chuỗi thứ hai là chỗ ngoại lệ ở mục 5 có thể xuất hiện.
Các bước	chạy ba cấu hình trên cùng dữ liệu: (a) SLAM thừa thuần (ORB-SLAM3 RGB-D), (b) SLAM thừa + dựng TSDF <i>sau</i> khi đã có tư thế, (c) một hệ dày dùng phần dư quang trắc dày để ước lượng tư thế; đo ATE, bộ nhớ, thời gian mỗi khung cho cả ba; rồi làm một loop closure và đo thời gian cùng chất lượng của việc cập nhật bản đồ dày.
Xong khi	bạn có một bảng ba cấu hình với bốn cột. Bạn phải quan sát được: (1) trên chuỗi có đặc trưng tốt, ATE của (a) và (c) gần nhau trong khi bộ nhớ chênh nhau hai bậc — xác nhận mục 5; (2) trên chuỗi nghèo đặc trưng, (c) có thể thắng rõ — xác nhận ngoại lệ; (3) chi phí cập nhật bản đồ dày sau loop closure lớn hơn bạn nghĩ, và đó là con số ít ai công bố.
Nối với	C/13-quan-ly-ban-do cho bài toán kích thước ở quy mô này; D/20-toi-uu-hieu-nang-va-he-thong cho ngân sách bộ nhớ thật trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thông tin về tư thế tập trung ở những chỗ *có cấu trúc*, và một front-end thừa tốt đã lấy gần hết những chỗ ấy. Theo Jacobian ở A/03, đóng góp vào ma trận thông tin tỉ lệ với gradient của hàm đo; pixel không có gradient đóng góp đúng bằng không. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết nhưng không thêm ràng buộc mạnh cho tư thế. Ngoại lệ: môi trường rất nghèo đặc trưng, nơi gradient yếu nhưng khác không và detector thừa không tồn tại được. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — ta có thể dùng phần dư dày mà vẫn chỉ lưu bản đồ thừa.
- A2. Nó cần một trường khoảng cách — ESDF — trả lời trực tiếp hai câu: điểm này có trống không, và cách chướng ngại gần nhất bao xa (kèm gradient, vì phần lớn bộ lập kế hoạch hiện đại dùng hàm chi phí theo khoảng cách). Một mesh đẹp không phải câu trả lời vì để tính khoảng cách tới chướng ngại từ mesh, bạn phải tìm tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ là quá đắt. Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*.
- A3. Ba vùng: (a) *không có vân* — mọi pixel trong một hàng giống hệt nhau nên không có thông tin xác định disparity; (b) *bị che khuất* — không tương ứng nào tồn tại; (c) *phản chiếu hoặc trong suốt* — tệ nhất, vì khớp *thành công* nhưng khớp với ảnh phản chiếu, cho độ sâu của một vật không có ở đó, tức hổng im lặng. Cả ba là giới hạn vật lý vì thông tin *không tồn tại trong dữ liệu* — không thuật toán nào tạo ra thông tin từ chỗ không có. Cái tốt nhất có thể làm với (a) là nội suy và *thừa nhận* rằng đó là nội suy; với (b) là đánh dấu “không biết”; với (c) thì chưa có lời giải tốt.

- A4. (1) *Truy vấn*: 3DGS tối ưu cho render, không cho việc trả lời “điểm này có trống không”; một đám Gaussian không phải hàm occupancy, và trích ESDF từ nó là một bước thêm không rẻ và không bảo đảm — với điều hướng đây là điểm chặn. (2) *Bộ nhớ và phần cứng*: hàng triệu Gaussian cộng yêu cầu GPU, ngoài ngân sách của một SoC nhúng. (3) *Tính dự đoán được*: chế độ hồng của TSDF đã hiểu rõ sau mười lăm năm, của 3DGS-SLAM thì chưa — cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07. Đánh giá của tôi tại 9/2026: 3DGS rất mạnh cho tái tạo ngoại tuyến, chưa phải lựa chọn cho robot điều hướng trên phần cứng hạn chế.
- A5. Bản đồ dày phải *biến dạng theo*. Với **TSDF** điều này khó vì dữ liệu đã được *hợp nhất* vào một lưới voxel cố định trong không gian, và không có cách nào “dịch” một lưới đã trộn dữ liệu từ nhiều góc nhìn — cách duy nhất đúng là tích hợp lại từ đầu, tức phải lưu toàn bộ ảnh độ sâu gốc. Các thỏa hiệp: chia thành khối gắn với keyframe (chấp nhận đường nối không khớp), hoặc chỉ tích hợp lại vùng ảnh hưởng nhiều nhất. Với **surfel** thì dễ hơn nhiều vì mỗi surfel gắn với một keyframe nên dịch theo tự nhiên — đó chính là lý do ElasticFusion chọn surfel. Đây là ví dụ rõ về việc lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai loại bản đồ. Một tấm bản đồ du lịch gấp bỏ túi, ghi vài chục địa điểm để nhận. Và một cái sa bàn mô phỏng cả thành phố tới từng bậc thềm.

Bản đồ du lịch đủ để bạn biết mình ở đâu. Sa bàn thì trả lời được những câu khác: cái xe đẩy của tôi có lọt qua chỗ này không, cái kệ kia có đủ chỗ đặt vật này không.

Và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn*. Để biết mình ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận ra. Cả một bức tường phẳng trong sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Nên câu hỏi đúng không phải “bản đồ nào tốt hơn” mà là “*tôi định dùng bản đồ để làm gì*”. Nếu để biết mình ở đâu, tấm gấp bỏ túi là đủ và rẻ hơn hàng trăm lần. Nếu để quyết định đi lối nào, bạn cần thứ khác — nhưng không phải sa bàn đẹp, mà là một thứ trả lời nhanh câu hỏi “chỗ này có đi được không”. Hai thứ đó không giống nhau: một cái nhìn giống thế giới, một cái trả lời nhanh câu hỏi bạn cần hỏi.

Còn một chuyện nữa, ít ai nghĩ tới. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ lại cho đúng, thì với tấm gấp bỏ túi bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải đập đi làm lại cả một khu — vì các mảnh đã dính chặt vào nhau và không dịch riêng được. Đó là lý do bản đồ chi tiết khó sống chung với một hệ thống hay phải sửa lại chính mình.

Chỗ không rút gọn được: chỗ không rút gọn được là các bề mặt phản chiếu. Một tấm gương hay một cửa kính cho hệ thống một phép đo *trông hoàn toàn hợp lệ* về một vật không tồn tại ở đó. Khác với vùng trống trơn — nơi hệ thống biết mình không biết — ở đây nó tự tin và sai. Chưa có lời giải tốt, và đó là lý do robot vẫn đâm vào cửa kính.

Một câu: *Xây bản đồ dày vì bạn cần trả lời một câu hỏi cụ thể, không vì bạn kỳ vọng nó giúp bạn biết mình đang ở đâu.*

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bản đồ dày không cải thiện độ chính xác tư thế* — suy từ cấu trúc Jacobian (cửa a) và kiểm được bằng mini project. Nó bị bác bỏ nếu một hệ dày vượt hệ thưa một cách nhất quán trên các chuỗi *có đặc trưng tốt* — tức ngoài ngoại lệ đã nêu. Tôi cho là không, nhưng cần nói rõ một giới hạn của lập luận: nó giả định front-end thưa đã *tốt*. Với một front-end thưa kém, bổ sung thông tin dày đúng là có ích, và lúc đó kết luận đúng là “*hãy sửa front-end*” chứ không phải “*hãy dùng dày*”.

Mệnh đề thứ hai — *ba vùng thất bại của stereo là giới hạn vật lý* — đúng theo định nghĩa cho hai vùng đầu (thông tin không tồn tại). Với vùng thứ ba (phản chiếu) thì *không hoàn toàn*: thông tin

có tồn tại — một hệ có thể nhận ra rằng đây là gương nếu nó suy luận về tính nhất quán của hình học phản xạ qua nhiều góc nhìn. Nên phát biểu chặt hơn: *với hình học hai khung thuần túy thì đó là giới hạn; với suy luận cấp cao hơn thì có thể phát hiện*. Đây là một hướng nghiên cứu chưa được khai thác nhiều.

Mệnh đề thứ ba — đánh giá về 3DGS tại 9/2026 — là một phán đoán về mức trưởng thành, và nó là phần dễ hết hạn nhất của cả cây. Nó bị bác bỏ bởi thời gian: nếu trong hai năm nữa có một hệ 3DGS chạy điều hướng trên SoC nhúng với ESDF trích xuất được tin cậy, đánh giá này sai. Tôi cho rằng khả năng đó không nhỏ, và đó là lý do khối *Cạm bẫy* ở mục 4 yêu cầu rà lại sau sáu tháng.

1.7 Kết nối

Nối lên trên. A/01-hình-hoc-da-thi cho triangulation và hình học stereo. A/06-bat-dinh-va-lan-truyen-sai-so cho quan hệ $\sigma_z \approx z^2 \sigma_d / (bf)$, thứ chỉ phối chất lượng của *mọi* bản đồ dày và giải thích vì sao bản đồ dày ở xa là bản đồ dày sai. B/08-mo-hình-phan-du cho phần dư quang trắc và cho lời cảnh báo về prior độ sâu học được, áp dụng nguyên vẹn ở mục 2B.

Nối ngang. C/13-quan-ly-ban-do là chương mà bài toán kích thước được đặt ra; ở đây nó gay gắt hơn hai tới ba bậc độ lớn, và vấn đề biến dạng khi loop closure ở mục 3B là một bài toán mà chương 13 không gặp. C/17-hoc-may-trong-slam bàn kỹ hơn về các mô hình sinh ra độ sâu và về việc hiệu chuẩn bất định của chúng.

Nối xuống dưới. D/20-toi-uu-hieu-nang-va-he-thong là chỗ ngân sách bộ nhớ và tính toán trở thành ràng buộc cứng — và bản đồ dày là thứ tiêu tốn ngân sách ấy nhiều nhất. D/22-huong-nghien-cuu-con-trong-xếp “biểu diễn neural cho localization thực dụng” làm một trong tám khoảng trống, với lý do nằm trọn trong mục 4B.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bản đồ dày thường không cho tư thế chính xác hơn? Trả lời bằng ngôn ngữ gradient và ma trận thông tin, rồi nêu ngoại lệ.
2. Bộ lập kế hoạch cần biểu diễn nào, và vì sao mesh không phải câu trả lời dù nó mô tả thể giới chính xác hơn?
3. Nêu ba vùng mà stereo thất bại, và nói vùng nào hỏng im lặng — tức nguy hiểm nhất.
4. Vì sao TSDF khó biến dạng sau loop closure còn surfel thì dễ? Nêu hai cách thoả hiệp cho TSDF.
5. Nêu ba lý do một robot sản phẩm có thể vẫn chọn TSDF thay vì 3DGS năm 2026, và nói lý do nào là điểm chặn cứng.
6. Một bản đồ dày trộn giữa phép đo và độ sâu học được. Vì sao đó là nguy hiểm cho ứng dụng an toàn, và cách xử lý đúng là gì?
7. Nếu mục tiêu là định vị chính xác hơn, nêu ba chỗ nên đầu tư thay vì xây bản đồ dày.

Nguồn

- [1] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [2] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [3] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [4] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [5] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [6] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [7] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [8] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [9] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.